Motors basats en tiles

Professors de VJ

Motors basats en tiles

- TBE (Tile Based Engines)
- Tile
  - Bloc fonamental d'un tauler.
  - Habitualment quadrat.
- Objectius
  - Estalviar memòria.
  - Reutilitzar gràfics.
  - Generació de contingut dinàmic.

Objectius

- Reutilitzar gràfics
  - Temps dels artistes és important (i car).
    - Artistes ↔ Conjunt de tiles
    - Dissenyador de nivells ↔ Mapes
  - Limitacions de tamany del binari
- Contingut dinàmic
  - Mapes generats “aleatoriament”.
    - Millor proceduralment.
  - Millora la rejugabilitat.

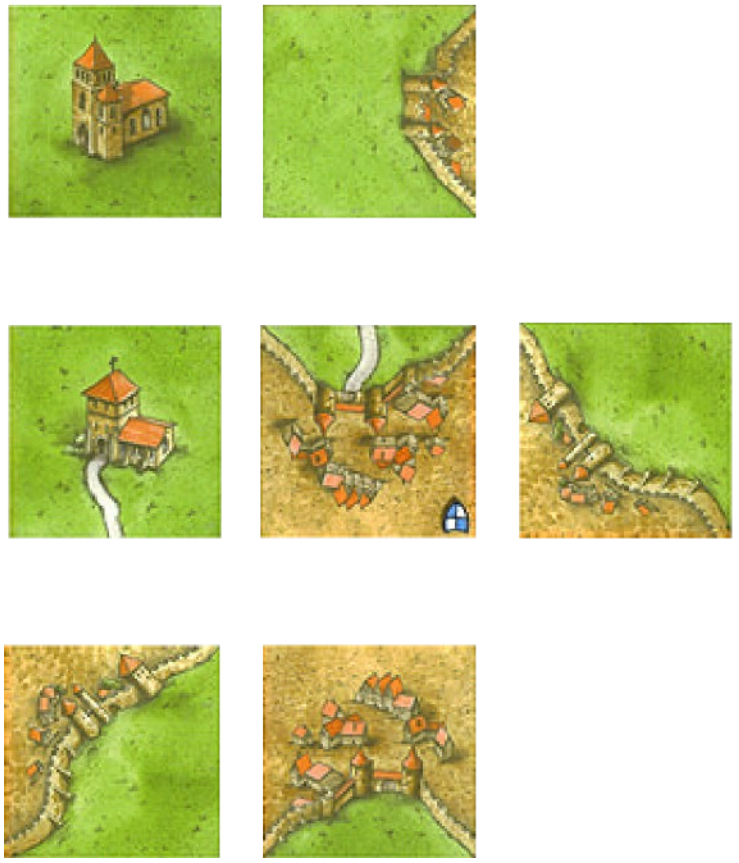
Tiles

A nivell formal:

- Imatge
- Amplada i alçada
  - Cal decidir a partir de:
    - Tamany de mapa
    - Restriccions de memòria
  - Habitualment potència de 2: 16, 32, 64, ...
    - Hardware antic: Operacions més simples.
    - Hardware nou: Textures potència de 2.

Motors basats en tiles

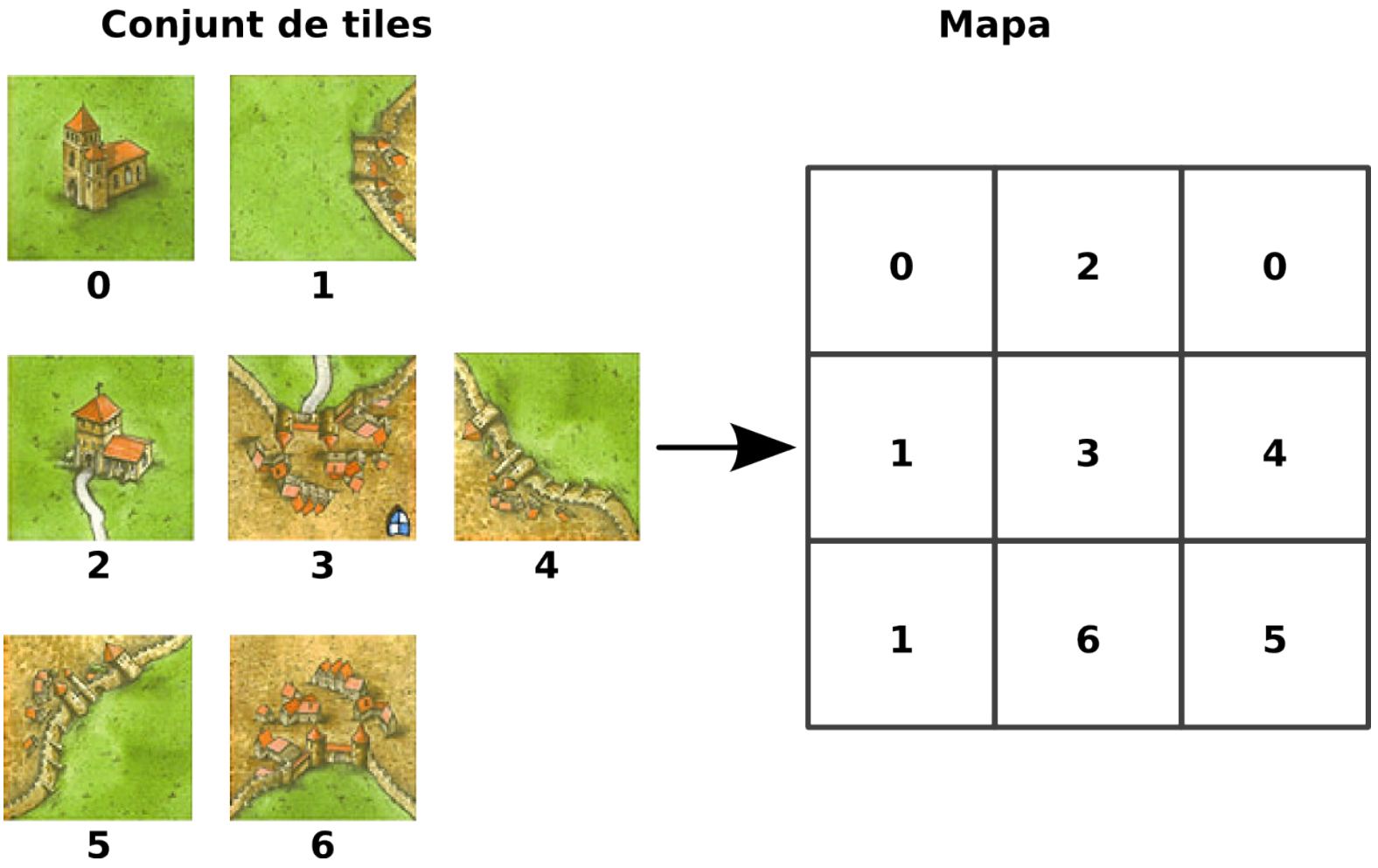
Conjunt de tiles



Mapa



Motors basats en tiles



Motors basats en tiles

Els tiles es poden agrupar en una sola textura.

Textura



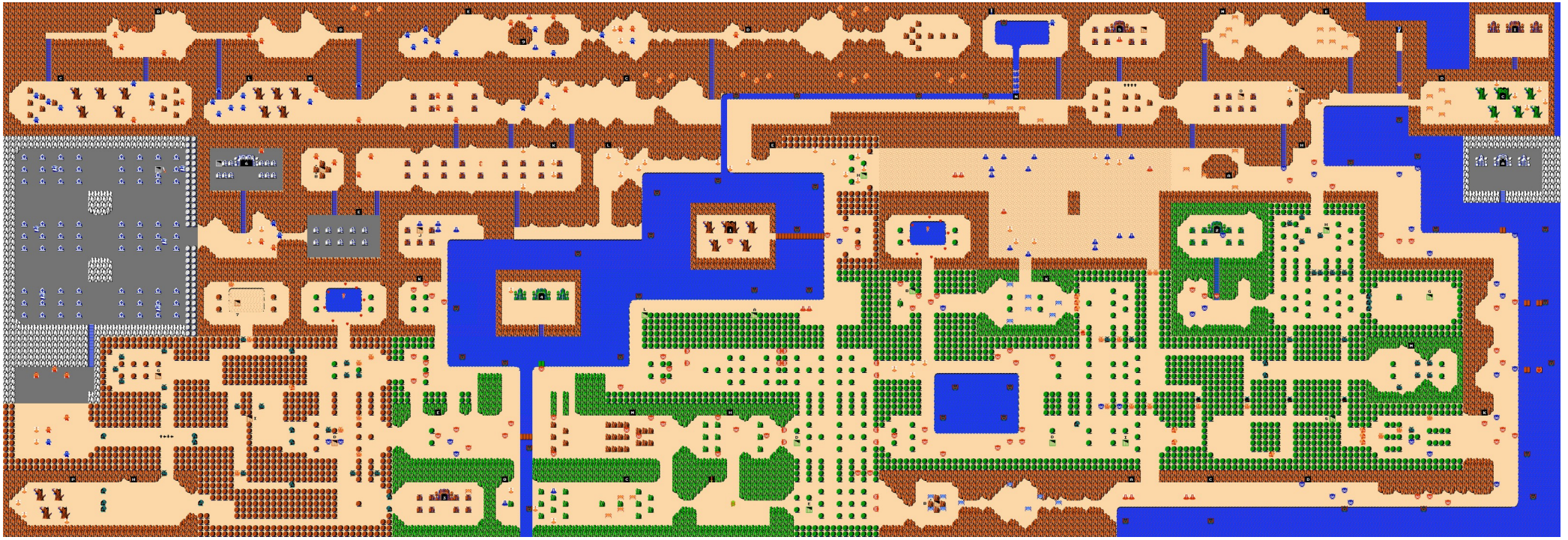
Mapa

0	2	0
1	3	4
1	6	5

Conjunt de tiles

0	1	2	3	4	5	6
x y	x y	x y	x y	x y	x y	x y
0 0	1 0	2 0	0 1	1 1	2 1	0 2

The Legend of Zelda (NES, 1986)

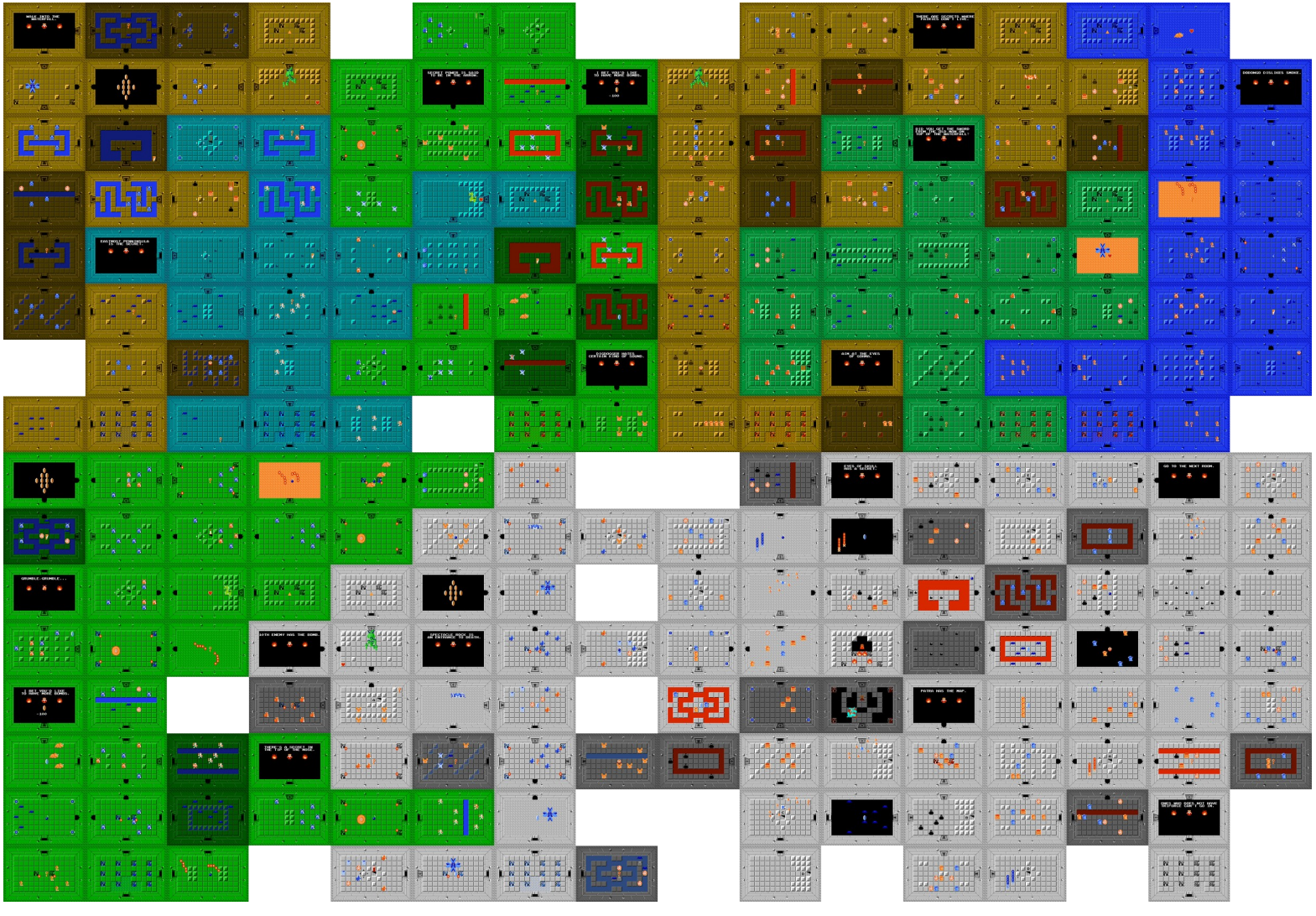


4096 x 1408 píxels

NES → Joc en ROM. Màxim 1 MB.

The Legend of Zelda (NES, 1986)

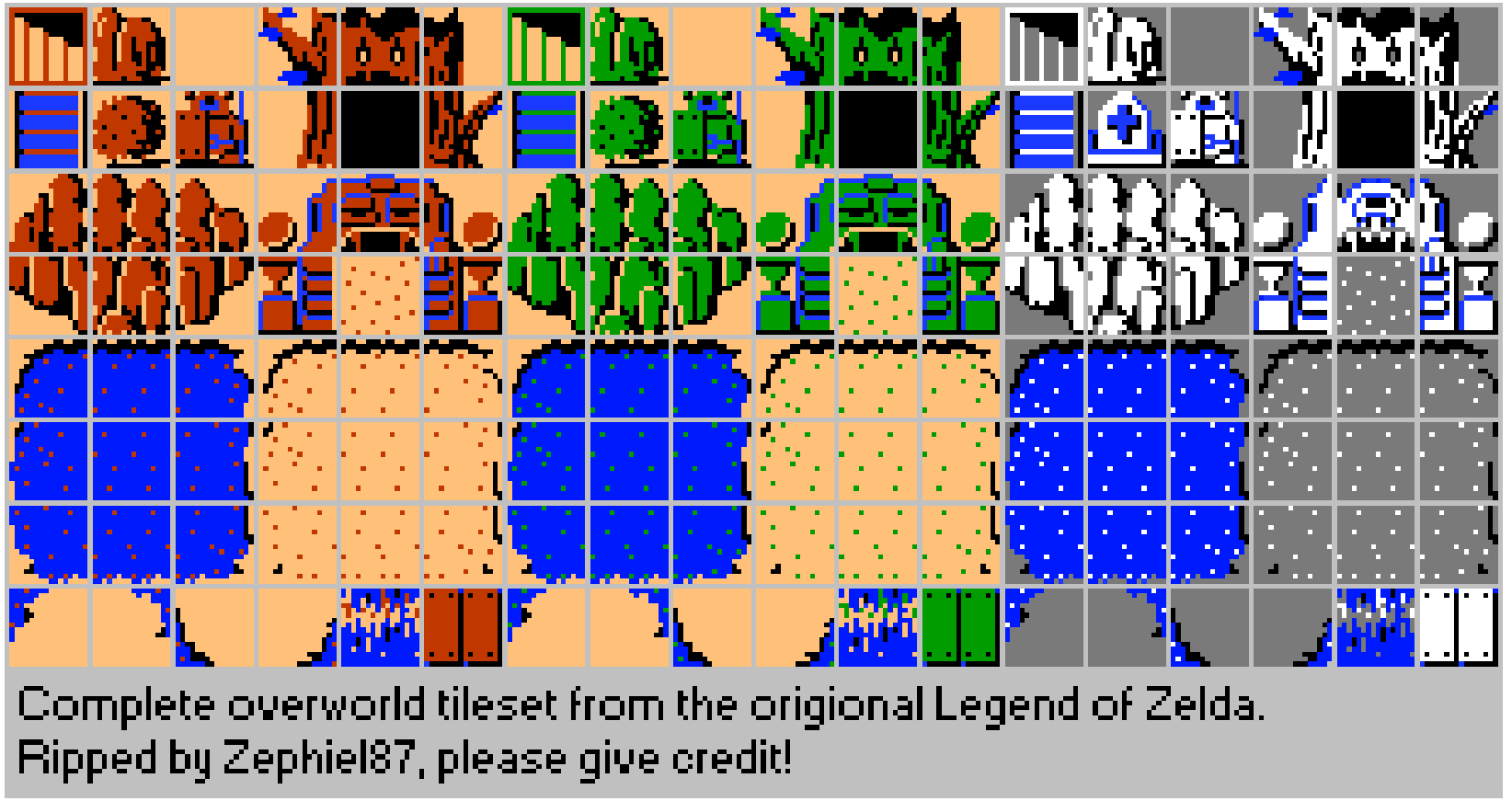
Encara n’hi havia més:



4096 x 2816

The Legend of Zelda (NES, 1986)

Tiles (Zelda Overworld)



18x8 = 144 Tiles de 16x16

The Legend of Zelda (NES, 1986)

Consum de memòria (suposant 4 bits/pixel)

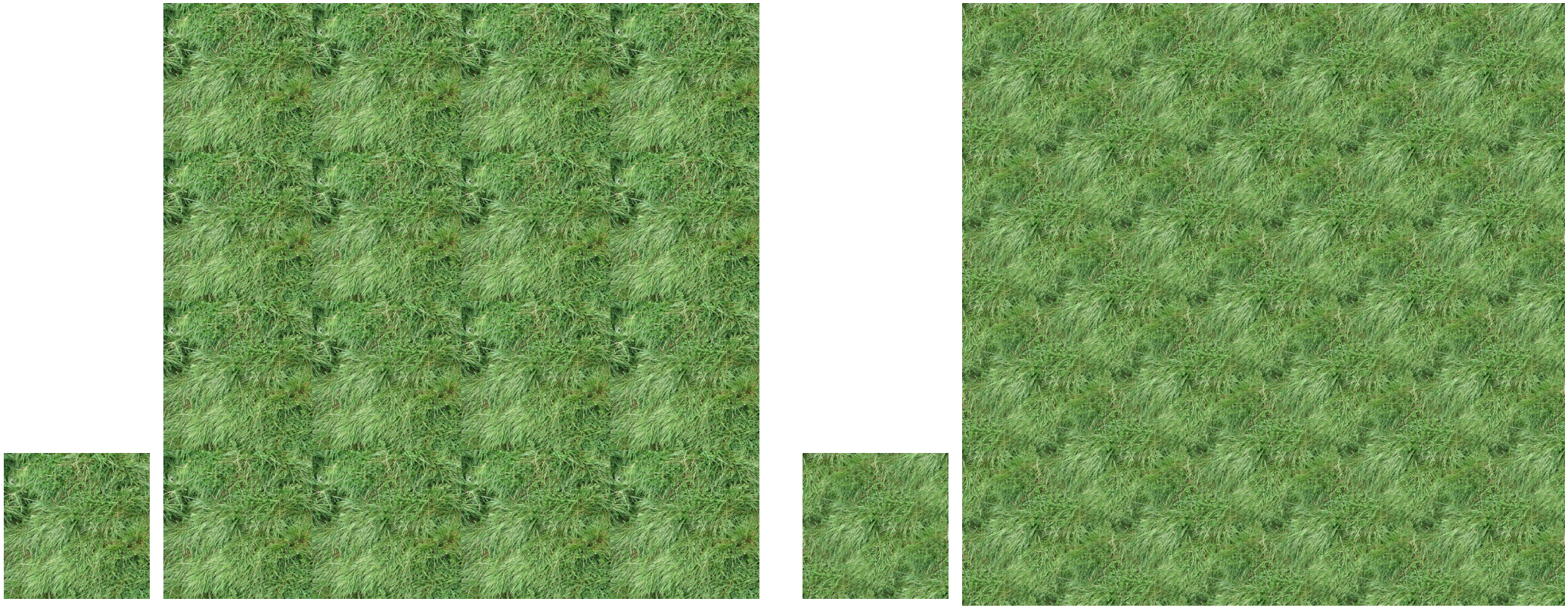
- Com a imatge:
  - 4096 x 1408 = 2.75 MB
- Com a mapa de tiles:
  - Joc de tiles: 18 x 8 tiles de 16 x 16 = 18 KB
  - Mapa: 256 x 88 = 22 KB
  - Total: 40 KB
- 70 vegades menys.

Tiles

Continuitat entre tiles:

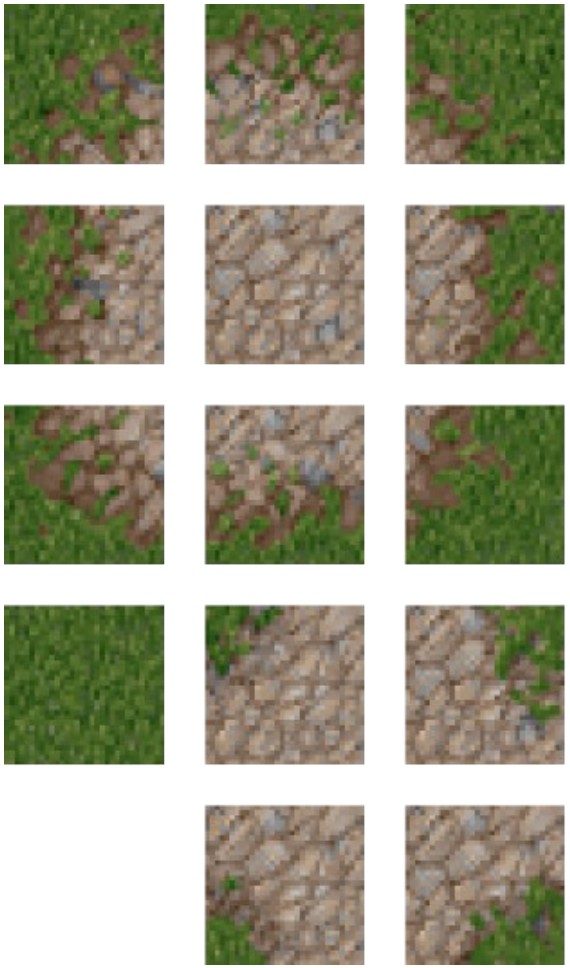
- Tiling textures
  - Textures que es poden repetir sense discontinuitat
  - També és útil en 3D.
- Tiles de connexió.
  - Entre qualsevol parella de terrenys.

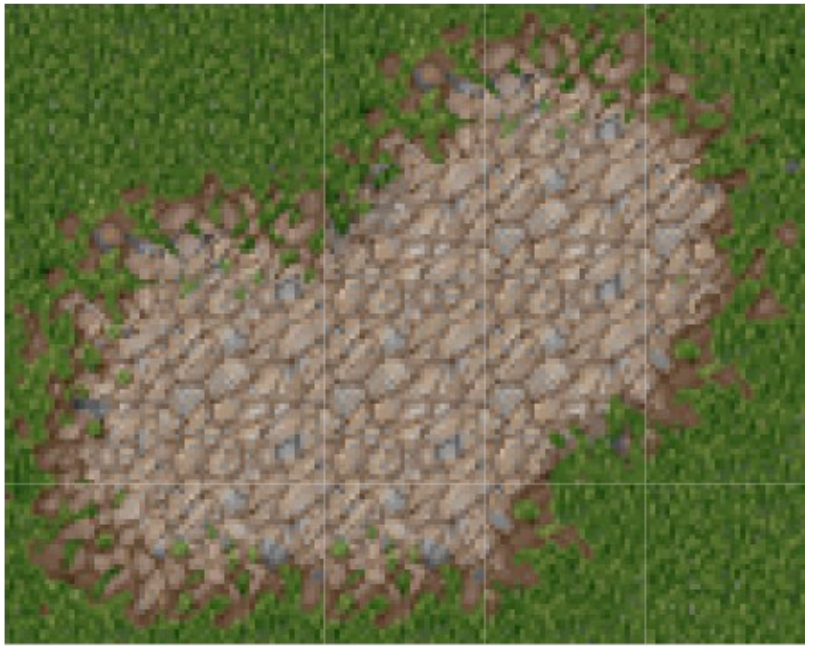
Tiling textures



Tiles

Tiles de connexió





Alternatives

- Imatges grans pels fons.
  - Més detall, però més espai ocupat.
- Combinació híbrida.
  - Mapa de tiles + Imatges grans.
    - Indexos poden fer referència a imatges més grans que el propi tile.
- Tenir més nivells:
  - Aprofitar repetició de tiles → Clusters
  - Ultima VII: Mapa de tiles + Clusters + Superclusters

Layers

La majoria de jocs necessiten objectes estàtics al mapa:

- Vegetació, Objectes, ...

No volem tenir un tile per a cada combinació de terra i objecte.

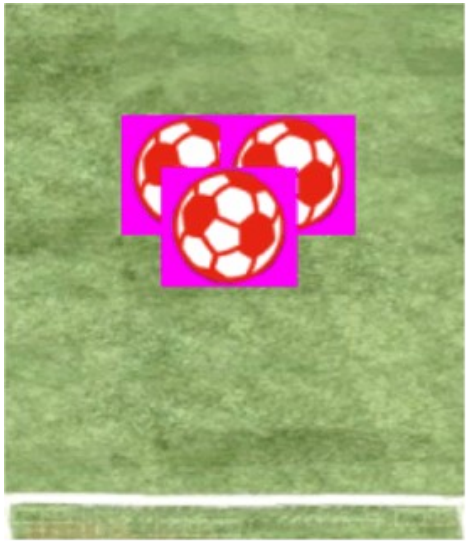
Solució:

- Múltiples capes (layers).

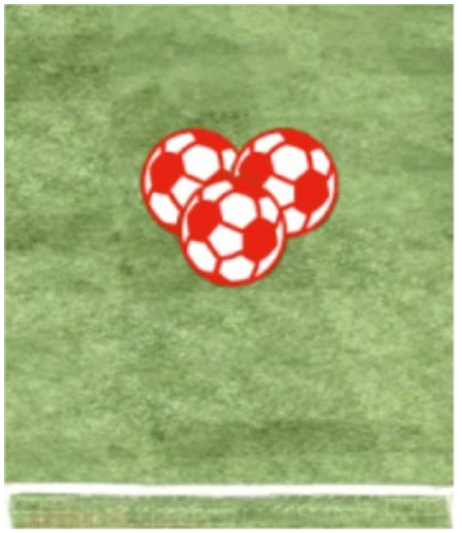
Transparència

Múltiples capes → Transparència

- Via color clau (color key)
  - Mètode antic.
  - Un color s'ignora quan es pinta.
- Via canal alpha (RGBA)
  - OpenGL i DirectX ho suporten directament.
  - Permet tranparència variable (A≠0 i A≠1).
    - Elimina el aliasing de la transparència.







Layers

Hi ha dos tipus de capes:

- Capes denses
  - Es fan servir com a fons.
  - S'implementen com a mapes de tiles.
- Capes disperses
  - Es fan servir per mostrar objectes.
    - Normalment estàtics.
  - S'implementen com un vector.

Capes denses

```
struct Tile
{
    int index;
    // Atributs del tile
    // Per exemple:
    bool bWall;
};
```

```
class DenseLayer
{
protected:
    vector<Tile> map;
    int mapWidth, mapHeight;
    int tileWidth, tileHeight;
    int depth;

public:
    void render();
};
```

```
void DenseLayer::render()
{
    int pos=0;
    for(int j=0; j<mapHeight; j++)
        for(int i=0; i<mapWidth; i++, pos++)
            RenderTile(map[pos].index, i*tileWidth, j*tileHeight,
                       tileWidth, tileHeight);
}
```

Capes denses

Caldria:

- RenderTile necessitarà accés a la classe que s'encarregui de la gestió de textures.
- Tenir en compte la possibilitat que una textura contingui més d'un tile.
- Visualitzar només aquells tiles que són visibles.
- Possibilitat de tiles grans (majors que widthTile, heightTile).
- ...

Capes disperses

- Els tiles d'una capa poden no cobrir tot el mapa.
- Adequat per tiles de tamanys variables.





Capes disperses

```
struct VariableTile
{
    int index;
    int x, y, depth;
    int width, height;
    // Atributs del tile
    // Per exemple:
    bool bWall;
};
```

```
class SparseLayer
{
protected:
    vector<VariableTile *> map;

public:
    void render();
};
```

```
void SparseLayer::render()
{
    vector<VariableTile *>::iterator itTile;
    for(itTile=map.begin(); itTile!=map.end(); itTile++)
        RenderTile((*itTile)->index, (*itTile)->x,
                   (*itTile)->y, (*itTile)->width,
                   (*itTile)->height);
}
```

Capes disperses

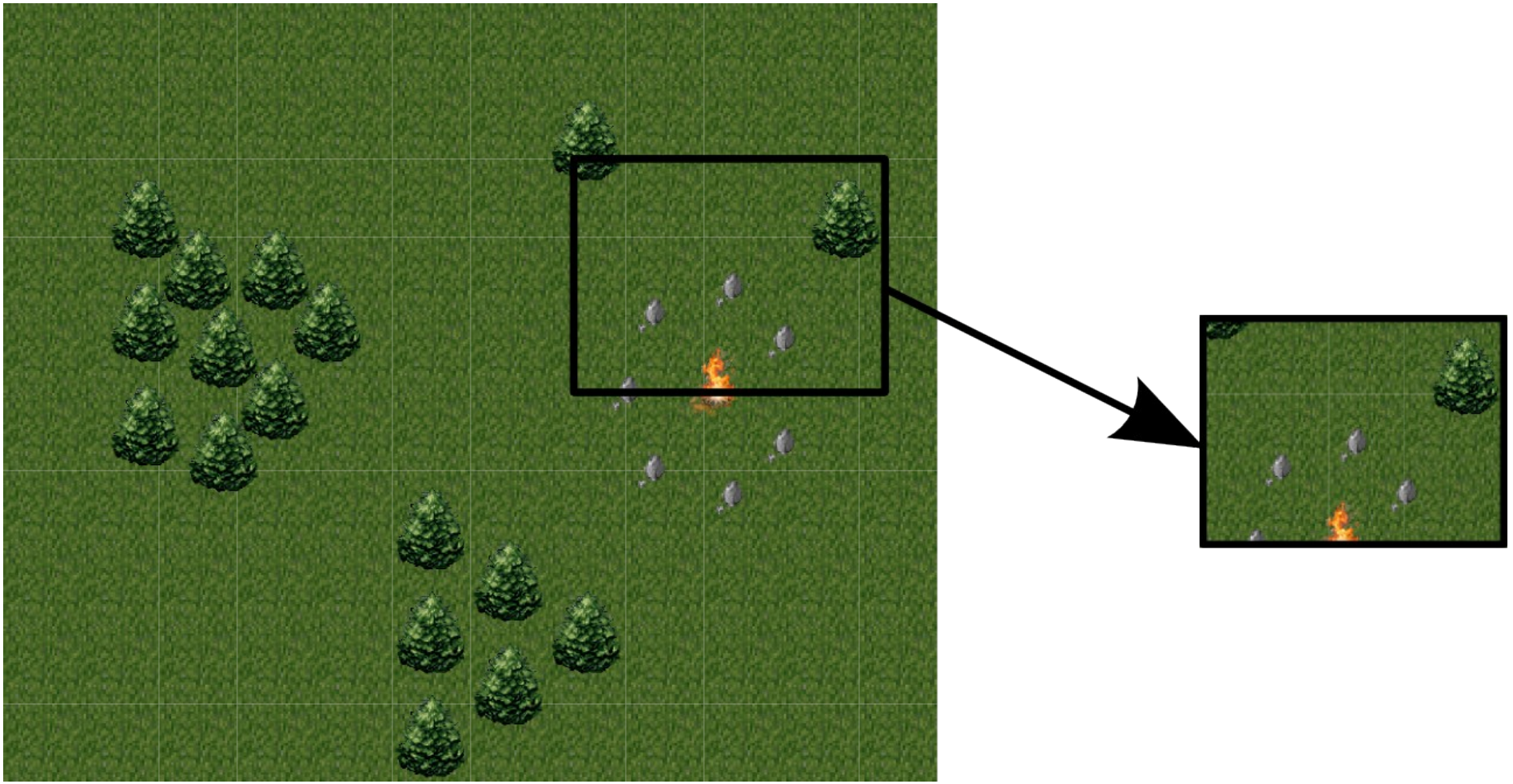
Com amb les capes denses caldria:

- RenderTile necessitarà accés a la classe que s'encarregui de la gestió de textures.
- Tenir en compte la possibilitat de que una textura contingui més d'un tile.
- Visualitzar només aquells tiles que són visibles.
- Possibilitat de tenir posicions no enteres.
- ...

Scrolling

- El món del joc pot ser molt més gran que la pantalla:
  - Divisió en pantalles o nivells.
    - Disminueix el temps de càrrega.
    - Disminueix el consum de memòria.
  - Scrolling
    - Veiem el món a traves d'una finestra.
      - Càmera

Scrolling



Scrolling

Caldrà moure la càmera per a seguir l'acció del joc:

- Posició i tamany de la càmera:
  - float cameraX, cameraY;
  - float cameraWidth, cameraHeight;
- cameraX no hauria de sortir del rang:
  - [0, mapWidth * tileWidth - cameraWidth]
- cameraY no hauria de sortir del rang:
  - [0, mapHeight * tileHeight - cameraHeight]
- Velocitat de la càmera:
  - Cal actualitzar el moviment de la càmera de forma independent de la velocitat del main loop.
  - També permet canviar la velocitat de scroll.
  - float cameraVx, cameraVy;

Scrolling

Implementació bàsica:

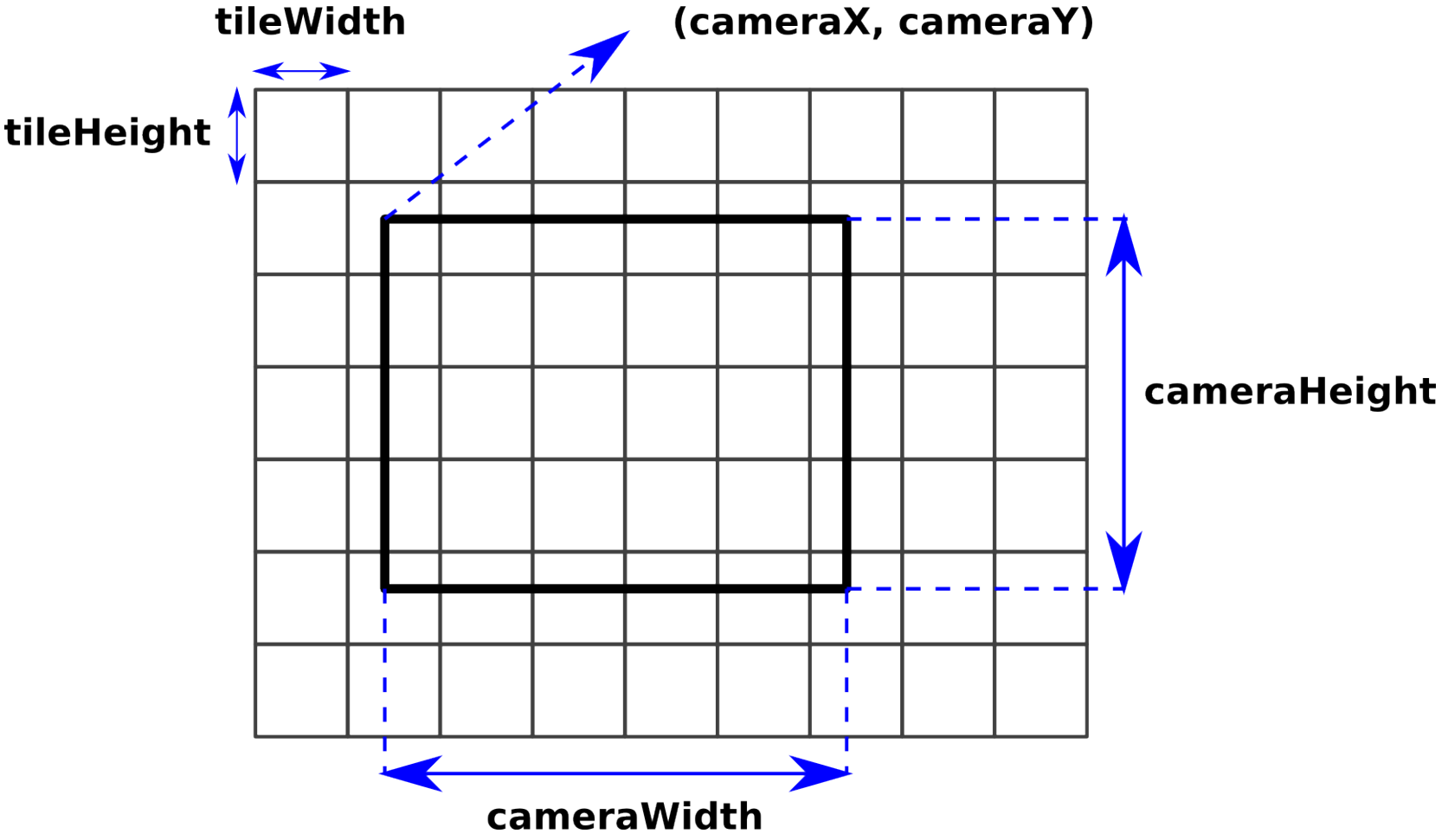
- Cal enregistrar les tecles de moviment (o les entrades adequades) → Fase d'Entrada
- Actualitzar la posició de la càmera → Fase d'Actualització

```
cameraX += loopTime * cameraVx
cameraY += loopTime * cameraVy
```

- Visualitzar només aquells tiles que són visibles → Fase de Render

Scrolling

Visualitzem només els tiles visibles:



Scrolling

$$min_Y = \frac{cameraY}{tileHeight}$$

$$min_X = \frac{cameraX}{tileWidth}$$



$$max_X = \frac{cameraX + cameraWidth}{tileWidth}$$

$$max_Y = \frac{cameraY + cameraHeight}{tileHeight}$$

Scrolling

- Es dibuixen els tiles entre els valors min_X, max_X, i entre min_Y i max_Y.
- Cal:
  - Desplaçar els tiles: (-cameraX, -cameraY).
  - O canviar el frustrum:
    - void gluOrtho2D(GLdouble left, GLdouble right, GLdouble bottom, GLdouble top);
    - gluOrtho2D(minX, maxX, minY, maxY);

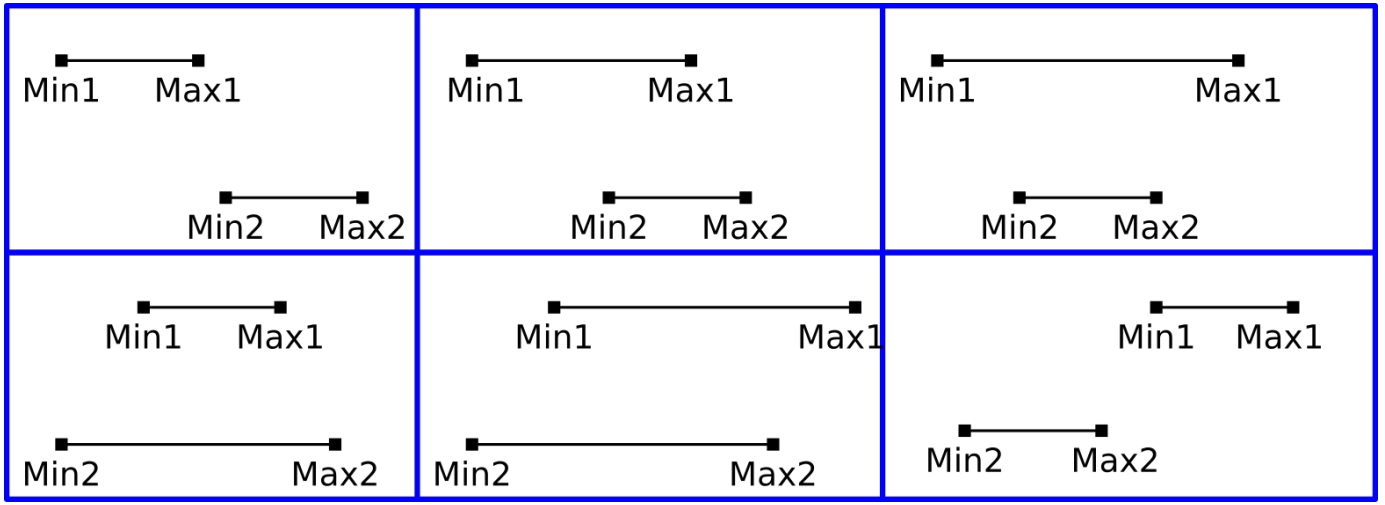
Scrolling

Com ens ho fem per una capa dispersa?

- Fer la intersecció entre la càmera i cada una de les capses dels tiles a la capa.
  - Intersecció de capses 2D.
- Es pot accelerar:
  - Graella regular.
  - Jerarquia de capses.

Scrolling

Intersecció de capses → Intersecció d'intervals 1D



Intersecció si es compleix que (totes dues):

- Min1 < Max2
- Min2 < Max1

Scrolling

2D

- Capsa 1: (minx1, miny1) – (maxx1, maxy1)
- Capsa 2: (minx2, miny2) – (maxx2, maxy2)

Intersecció si és compleix (totes dues):

- (minx1 < maxx2) && (minx2 < maxx1)
- (miny1 < maxy2) && (miny2 < maxy1)

Scrolling

Pasos:

- Posició i velocitat de l'avatar del jugador:
  - float jugadorX, jugadorY;
  - float jugadorVx, jugadorVy;
  - Cal que la velocitat vingui determinada per l'entrada.
- Posició de la càmera:
  - No s'actualitza amb la seva velocitat.
  - Fa servir el moviment del jugador.

Scrolling

Problema: Estem movent la càmera directament.

- Depèn del gènere
  - En jocs de estratègia funciona, però ...
  - Normalment la càmera depèn de l'avatar del jugador.
- En el cas dels plataformes (side-scrollers)
  - Cal seguir al jugador.
    - Amb l'avatar centrat.
      - Actualitzem la posició del jugador de la mateixa manera que ho fem amb la càmera.
    - Amb l'avatar dintre d'un cert subconjunt de la càmera.
      - Més complexe.

Scrolling

```
jugadorX += loopTime * jugadorVx;
jugadorY += loopTime * jugadorVy;
if((jugadorX – cameraX) < (cameraWidth / 3))
       cameraX = jugadorX - cameraWidth / 3;
if(((jugadorX – cameraX) > (2 * cameraWidth / 3))
       cameraX = jugadorX – 2 * cameraWidth / 3;
if((jugadorY – cameraY) < (cameraHeight / 3))
       cameraY = jugadorY - cameraHeight / 3;
if(((jugadorY – cameraY) > (2 * cameraHeight / 3))
       cameraY = jugadorY – 2 * cameraHeight / 3;
```

Inconvenient

- La càmera mostra menys del costat cap a on volem anar.

Solució

- La càmera es comença a moure quan ens apropem a un costat.
- Es mou amb una acceleració addicional.

Scrolling

```
jugadorX += loopTime * jugadorVx;
jugadorY += loopTime * jugadorVy;
if(jugadorVx > 0)
{
    if(((jugadorX – cameraX) > (cameraWidth / 3))
        cameraVx = FACTOR * jugadorVx;
    else
        cameraVx = 0;
}
else
{
    if((jugadorX – cameraX) < (2 * cameraWidth / 3))
        cameraVx = FACTOR * jugadorVx;
    else
        cameraVx = 0;
}
cameraX += loopTime * cameraVx;
cameraX = max(cameraX, jugadorX - 2*cameraWidth/3);
cameraX = min(cameraX, jugadorX - cameraWidth / 3);
// Anàleg per Y
```

Cal que FACTOR > 1

És molt millorable:

- Moviment accelerat.
- Deceleració quan ens apropem a l'objectiu.